

TECHNICAL DESIGN DOCUMENT

Kafka-Based Event Streaming for Workflow Lifecycle Events

Partitioning · Offset Management · Idempotency · Consumer-Rebalance Stability

Explained in plain English, component by component, with a full end-to-end walkthrough and annotated architecture diagrams.

Contents

1. The Project in Plain English
2. Vocabulary — Every Term in One Line
3. The Components, One by One
4. The Workflow, End to End
5. Architecture Diagram (Figure 1) and Its Walkthrough
6. Consumer Internals (Figure 2) and Its Walkthrough
7. The Rebalance Problem and How It Was Solved
8. Configuration Reference
9. Design Rules Worth Remembering

1. The Project in Plain English

1.1 The one-paragraph version

A workflow is a job made of many steps — for example, onboarding a customer, processing a claim, or running a nightly batch. As that job moves along, it keeps producing news about itself: it started, step three finished, it failed, someone cancelled it, it completed. That news is what we call a lifecycle event. This project built the pipe that carries those events. Instead of every service calling every other service directly, the services that know something write one small message onto a shared, durable log called Kafka, and every service that cares reads from that log at its own pace. The work then focused on four hard parts: putting related events in the right lane so their order is preserved (partitioning), remembering exactly how far each reader has got (offset management), making sure a message that arrives twice only takes effect once (idempotency), and stopping the readers from constantly kicking each other off their lanes (the consumer-rebalance problem).

1.2 A simple analogy

Picture a very long paper receipt roll that only ever gets added to at the bottom. Every time something happens to a workflow, one line is printed on that roll. Nobody erases anything; the roll just grows.

- **The roll is the Kafka topic.** It is the permanent record of everything that happened, kept for a fixed number of days.
- **There is not one roll but four, side by side.** These are the partitions. Which roll a message is printed on is decided by the workflow's ID, so every event about workflow #77 always lands on the same roll and therefore stays in the correct order relative to itself.
- **Readers stand at the rolls with a bookmark.** The bookmark is the offset — the line number they have finished. If a reader goes home and a colleague takes over, the colleague picks up from the bookmark rather than starting again from line one.
- **Readers only move the bookmark once the work is genuinely done.** This is commit-after-success. It means a crash can cause a few lines to be read twice, but never causes a line to be skipped.
- **Each reader keeps a list of line IDs already acted on.** This is idempotency. Reading a line twice is harmless because the second time it is recognised and ignored.

1.3 What was broken before, and what changed

Aspect	Before this project	After this project
How services talked	Direct synchronous calls. If the receiving service was down or slow, the caller was stuck or the update was lost.	Producers write once to Kafka and move on. Consumers read when they are ready. Neither side has to be up at the same time.
Ordering	Two updates for the same workflow could be applied out of order, leaving the wrong final state.	All events for one workflow share a partition, so they are consumed strictly in the order they happened.
Duplicates	A retry after a timeout could apply the same state change twice.	Each event carries a unique ID that is checked before the work is done, so a repeat is a no-op.

Aspect	Before this project	After this project
Losing progress	A crash mid-processing could leave a record neither finished nor retried.	Offsets are committed only after the work succeeds, so anything unfinished is simply redelivered.
Stability under load	Consumers were repeatedly declared dead and their partitions reassigned. The group spent its time rebalancing instead of working.	A fast poll loop and a bounded worker pool keep the consumer responsive, so the group stays stable and throughput is predictable.
Bad messages	One unprocessable message stalled everything behind it on that partition.	It is retried, then routed to a dead letter queue, and the rest of the partition keeps flowing.

In one sentence: the project turned a fragile web of direct service calls into a durable, ordered, replayable event stream that keeps working correctly even when parts of the system crash, restart, or fall behind.

2. Vocabulary — Every Term in One Line

Every term used later in this document, defined once here in ordinary language.

Term	What it means, simply
Event	A small message describing one thing that happened, such as <code>WORKFLOW_STARTED</code> or <code>STEP_FAILED</code> . It is a statement of fact about the past, not an instruction.
Lifecycle event	An event about the stages a workflow passes through: created, started, step completed, failed, retried, cancelled, completed.
Producer	Any service that writes events into Kafka.
Consumer	Any service that reads events out of Kafka and acts on them.
Broker	One Kafka server. A cluster of brokers stores the data and survives the loss of individual machines.
Topic	A named stream of events, like a labelled folder. Here it is <code>workflow.lifecycle.events</code> .
Partition	One of the parallel lanes a topic is split into. Order is guaranteed inside a lane, not across lanes.
Key	The value used to decide which lane a message goes to. Here it is the workflow ID.
Offset	The line number of a message inside its partition. It never changes and never goes backwards.
Commit	Saving the statement "my group has finished everything up to offset N on this partition" back into Kafka.
Consumer group	A team of consumer instances sharing one workload. Kafka gives each partition to exactly one member of the team.
Rebalance	The reshuffle that happens when a member joins, leaves, or is presumed dead, and partitions are handed out again.
Poll	The call a consumer makes to fetch the next batch of messages. It also doubles as proof that the consumer is alive.
Worker pool	A fixed set of threads that perform the actual processing, kept separate from the thread that polls.
Idempotency	The property that doing something twice has the same effect as doing it once.
Consumer lag	How many messages have been written but not yet processed. The main health metric of the pipeline.
Dead letter queue (DLQ)	A side topic where messages that can never be processed are parked for humans to inspect.
Replication factor	How many copies of each partition exist on different brokers, so data survives a broker failure.

3. The Components, One by One

Each component below is described in the same four ways: what it is, why the system needs it, how it actually works, and what would go wrong if it were removed. Read in order, they build the whole system from the front to the back.

3.1 Producer services — where events are born

What it is. The applications that already know something happened: the workflow orchestrator that drives a job forward, the task executor that runs individual steps, and the admin or API layer where a human cancels, pauses or retries a workflow. Each of these publishes an event at the moment the fact becomes true.

Why it is needed. The service that performs a state change is the only one that can honestly report it. Publishing at that exact moment means the record and the reality never drift apart.

How it works. The producer writes the workflow state to its own database and publishes the matching event as part of the same logical unit of work. In practice the event is first written to an outbox table inside the same database transaction, and a small relay reads that table and pushes to Kafka. That removes the classic failure where the database commits but the message is lost, or the message is sent and then the database rolls back.

Without it. Downstream systems would have to poll the workflow database on a timer, which is slower, heavier, and still misses short-lived states such as a step that started and finished between two polls.

3.2 The event envelope — the shape of a single message

What it is. The agreed structure every event follows, so that any consumer can understand any event without private knowledge of the producer.

```
{
  "eventId":      "5f2a-9c11-...",    // unique for this event, used for idempotency
  "eventType":    "STEP_COMPLETED",  // what happened
  "workflowId":   "wf-77",           // also used as the Kafka key
  "stepId":       "step-3",
  "occurredAt":   "2026-09-01T10:14:22Z",
  "schemaVersion": 2,                // lets old and new consumers coexist
  "producer":     "task-executor",
  "correlationId": "req-8842",        // ties the event back to the original request
  "payload":      { "durationMs": 1180, "result": "OK" }
}
```

Why it is needed. Two fields do the heavy lifting. The eventId is what makes duplicate detection possible at all — without a stable unique identifier there is no way to recognise a message you have already seen. The schemaVersion is what makes it safe to change the format later, because a consumer can read an older or newer version without crashing.

How it works. The eventId is generated once by the producer and never regenerated on retry, which is essential: if a retry produced a new ID, the duplicate would look like a brand-new event and would be applied twice. Fields are only ever added, never removed or repurposed, so old consumers keep working.

Without it. Every consumer would parse messages slightly differently, duplicates would be undetectable, and any format change would require every service to be redeployed at the same moment.

3.3 Partitioning and the key — how order is preserved

What it is. The topic is split into four partitions. Which partition a message lands on is decided by hashing its key, and the key here is the workflowId.

Why it is needed. Kafka guarantees order inside a partition, and only inside a partition. Ordering matters enormously for lifecycle events: applying COMPLETED before STARTED would leave a workflow in a nonsensical state. By making the key the workflow ID, every event about one workflow is forced into the same lane and therefore into the correct order.

How it works. Kafka computes $\text{hash}(\text{workflowId}) \bmod 4$ and sends the record to that partition. The calculation is deterministic, so workflow wf-77 goes to the same partition today, tomorrow and after a restart. Events for different workflows land in different lanes and are processed in parallel, which is exactly what we want — we need order per workflow, not order across the whole system.

Choosing the number of partitions. Partition count is the ceiling on parallelism: a group can never have more actively working consumers than partitions, because a partition is owned by exactly one consumer. Four partitions therefore allow up to four consumers to work at once. Partitions can be increased later but not decreased, and increasing them changes which lane a key maps to, so the number is chosen with headroom from the start.

Watch out for: an uneven key distribution. If one workflow is enormously busier than the rest, its partition becomes a hot lane while the others idle. The fix is a more granular key or a composite key, not more partitions.

Without it. Either everything goes into one lane and throughput is capped at a single consumer, or messages are spread at random and the ordering guarantee disappears entirely.

3.4 The topic, brokers and replication — durable storage

What it is. The topic workflow.lifecycle.events, physically stored across three brokers with each partition replicated three times and retained for seven days.

Why it is needed. This is what makes the stream a record rather than a transmission. Because messages are kept after being read, a consumer that was down for an hour can catch up, a new consumer can be added and replay history to build its own view, and a bug fixed on Tuesday can be re-run over Monday's events.

How it works. Each partition has one leader broker that handles reads and writes, plus two followers that copy it. The producer is configured with `acks=all`, which means a write is only acknowledged once the replicas have it, so losing a broker does not lose data — a follower is promoted and consumers carry on. Reading a message does not delete it; messages age out on the retention clock instead.

Without it. The stream would behave like a traditional queue where a consumed message is gone forever. Replay, late-joining consumers and recovery from processing bugs would all be impossible.

3.5 Offsets and the consumer-offsets topic — the bookmark

What it is. Every message in a partition has a permanent sequence number called an offset. For each consumer group Kafka stores, in an internal topic, the offset the group has committed on each partition.

Why it is needed. Kafka does not track which consumer has seen what. The consumer group tracks its own position, and the committed offset is the only thing that survives a crash, a restart, or a partition changing hands. It is the single source of truth for "where do we resume".

How it works. Three positions must be kept apart in your head:

Position	Meaning	Who moves it
Log end offset	The newest message written to the partition.	Producers, whenever they publish.
Current position	The next offset this consumer will read in memory. Moves the instant a batch is fetched.	The consumer, on every poll.
Committed offset	The last offset the group has durably declared as finished. This is the recovery point.	The consumer, only after processing succeeds.

The gap between the log end offset and the committed offset is the consumer lag, and it is the number to alarm on: flat lag means healthy, steadily rising lag means the consumers cannot keep up.

Without it. A restarted consumer would have to choose between reprocessing the entire seven-day history or skipping straight to the newest message and silently losing everything in between.

3.6 The consumer group — sharing the work

What it is. Several identical consumer instances that all declare the same group ID, workflow-lifecycle-cg. Kafka treats them as one logical reader and divides the partitions between them.

Why it is needed. It gives horizontal scaling and failover with no coordination code of your own. Start another instance and the load redistributes automatically; kill one and its partitions are picked up by the survivors.

How it works. One broker acts as the group coordinator. Members send heartbeats to it. The coordinator assigns each partition to exactly one member — never two, which is what prevents the same event from being processed concurrently by two machines. With four partitions and three instances the split might be 2-1-1. Adding a fourth instance makes it 1-1-1-1. A fifth instance would sit idle, holding no partitions, because there is nothing left to give it.

Key rule: the useful number of consumer instances is capped by the partition count. Scaling past it adds standby capacity, not throughput.

Without it. You would either run a single consumer and be limited to its speed, or run several independent consumers that all read everything and duplicate every side effect.

3.7 The lightweight poll loop — the heart of the fix

What it is. A single thread that does almost nothing except call poll(), hand the returned records to the worker pool, commit finished offsets, and immediately poll again.

Why it is needed. This is the direct fix for the rebalance problem, and it rests on a detail that surprises most people the first time: in Kafka, calling poll() is not only how you fetch data, it is also how you prove you are still working. If a consumer takes longer than max.poll.interval.ms between two poll calls, the coordinator concludes it has died and gives its partitions to somebody else.

The failure this replaced. The original loop fetched a batch and then processed every record inline before polling again. On a slow batch — a sluggish database, a third-party call, a large payload — the gap between polls exceeded the limit. The consumer was declared dead while it was in fact busy and healthy. Its partitions were reassigned, which triggered a rebalance, which stopped the other consumers too, which made them slower, which caused the next

eviction. The group spent most of its time reshuffling rather than processing, and because work in progress was abandoned mid-flight, the same records were processed again by the new owner.

How it works now. The loop keeps a small, fixed budget of work. `max.poll.records` caps each batch at 100. Handing a batch to the pool is a fast, non-blocking operation. The loop therefore returns to `poll()` in milliseconds regardless of how slow the actual processing is, so the consumer always looks alive because it genuinely is responsive. Meanwhile a separate heartbeat thread keeps the group membership fresh independently.

```
while (running) {
    records = consumer.poll(200ms);           // fast, always returns quickly
    for (record : records)
        pool.submit(record);                 // hand off, do not process here
    offsets = tracker.watermarkPerPartition();
    if (!offsets.isEmpty())
        consumer.commitSync(offsets);        // only offsets that truly finished
}
```

Without it. The consumer group returns to the rebalance storm: constant reassignment, repeated reprocessing, and throughput that collapses exactly when load is highest.

3.8 The bounded worker pool — controlled parallelism

What it is. A fixed number of worker threads, each drawing from a queue of limited size, doing the real processing that the poll loop deliberately refuses to do.

Why bounded is the important word. An unbounded pool or an unbounded queue would accept work faster than it could complete it. Memory would fill with pending records, garbage collection pauses would lengthen, and the process would eventually be killed — taking every uncommitted record with it. A bound converts an invisible memory problem into a visible, safe slowdown.

How it works. The pool has a fixed thread count sized to the real bottleneck, which is usually the database connection pool rather than the CPU. The queue in front of it also has a fixed capacity. When that queue reaches capacity, the poll loop calls `consumer.pause()` on the affected partitions. Paused partitions return no records on the next poll, but the poll call still happens, so the consumer remains alive in the group and is not evicted. When the queue drains below its threshold the loop calls `resume()` and records start flowing again. This is back-pressure: the system slows down on purpose instead of falling over.

Design choice	What it prevents
Fixed thread count	Thread explosion, context-switch thrash, and overwhelming the database with more concurrent queries than it has connections.
Bounded queue	Unbounded memory growth and the out-of-memory crash that follows it.
Pause and resume rather than blocking the loop	Being evicted from the group. Blocking the loop would stop polling, which is precisely the original bug.
Per-partition ordering inside the pool	Two events for the same workflow being processed at the same time in the wrong order.

Subtle but essential: records from the same partition must not be reordered by the pool. Work is dispatched so that one partition's records are handled by one worker at a time — usually by routing records to a worker slot chosen by partition. Without this, the parallelism you added would quietly destroy the ordering that partitioning was introduced to guarantee.

3.9 Offset management and commit-after-success

What it is. The rule that the committed offset is advanced only after the work for those records has genuinely completed, together with the bookkeeping needed to know which offsets those are.

Why it is needed. The order of these two operations decides the failure mode of the entire system, and there are only two options:

Order of operations	What happens on a crash	Verdict
Commit first, then process (at-most-once)	The offset says the record is done, but the work never ran. The event is lost permanently and nothing will ever retry it.	Unacceptable. A lost STEP_FAILED event means a workflow silently hangs forever.
Process first, then commit (at-least-once)	The work ran but the offset was not saved, so the record is delivered again after restart.	Correct choice. The only cost is occasional duplicates, and idempotency removes that cost.

How the watermark is calculated. Because a pool processes records concurrently, they finish out of order, and an offset commit is a statement about everything up to that point — not about one record. The consumer therefore tracks, per partition, which offsets have completed, and commits the highest offset that has no unfinished offset below it. That value is the watermark. If offsets 10, 11, 12 and 14 are done but 13 is still running, the group may only claim 12 and commits 13 as its next read position. Offset 14 will be redelivered after a crash, and the idempotency check will absorb it.

Practical details. Auto-commit is switched off entirely (`enable.auto.commit=false`), because a background timer that commits on a schedule has no idea whether the work finished. Commits are batched once per poll cycle rather than per record, since each commit is a network round trip. A synchronous commit is used at shutdown and when partitions are revoked, where correctness matters more than speed.

Without it. Either events are silently lost on every crash, or the same records are replayed from far too early, depending on which side you got wrong.

3.10 Idempotency — making a repeat harmless

What it is. A check performed before the work, keyed on the event's unique `eventId`, ensuring that processing the same event a second time changes nothing.

Why it is needed. At-least-once delivery makes duplicates a certainty, not a possibility. They arrive after a crash between processing and commit, after a rebalance that revokes a partition mid-batch, and after a producer retries a send whose acknowledgement was lost in the network. Idempotency is what turns "delivered at least once" into "applied exactly once", which is the behaviour the business actually needs.

How it works. A processed-events table or Redis set is keyed on `eventId` with a unique constraint and a time-to-live comfortably longer than the topic retention. Before doing anything, the consumer asks whether that ID is present. If it is, the record is marked complete and skipped without side effects. If not, the business change and the insertion of

the eventId happen inside one database transaction, so the two can never disagree — you cannot end up having done the work but forgotten to record that you did, or vice versa.

```
BEGIN TRANSACTION
  INSERT INTO processed_events(event_id) VALUES (:eventId)  -- unique key
  -- if this raises a duplicate-key error, the event was already handled:
  -- roll back, skip the record, mark the offset as complete
  UPDATE workflow SET state = :newState WHERE id = :workflowId
  INSERT INTO workflow_audit(...)
COMMIT
```

A second layer. Many state transitions are made naturally idempotent as well, by writing the target state rather than nudging it. Setting state = COMPLETED can be repeated safely; incrementing a counter cannot. Where an increment is unavoidable, the eventId check is the only defence, which is why it is applied first.

Without it. Every crash and every rebalance would produce duplicate notifications, double-counted metrics, and repeated state transitions — the exact class of bug that is hardest to reproduce and hardest to explain.

3.11 The rebalance listener — a clean handover

What it is. A callback Kafka invokes just before a consumer loses partitions and just after it gains them, used to hand over work tidily instead of dropping it.

Why it is needed. Rebalances are normal — every deployment causes one. Without a listener, a consumer losing a partition abandons whatever was in flight and never commits it, so the new owner replays a large, unpredictable amount of work. With one, the loss is small and bounded.

How it works. On partitions revoked, the consumer stops accepting new records, waits for in-flight work on those partitions to drain (with a timeout so shutdown cannot hang), commits the resulting watermark synchronously, then clears its tracking state and releases. On partitions assigned, it reads the committed offset for the new partitions and resumes from precisely that point, initialising fresh tracking state. Cooperative sticky assignment is used so that only the partitions that genuinely need to move are moved, and consumers keep the ones they already had rather than everybody stopping and restarting from scratch.

Without it. Every routine deployment would cause a burst of duplicate processing and a visible spike in lag.

3.12 Retry topic and dead letter queue — containing failure

What it is. A tiered failure path: retry in place for brief problems, a delayed retry topic for longer ones, and a dead letter queue for messages that will never succeed.

Why it is needed. A partition is strictly ordered, so a record that cannot be processed and is never committed blocks everything behind it. One malformed message could freeze a quarter of the pipeline indefinitely. The tiers exist to distinguish a problem that will go away from one that will not.

Tier	For what	Action
In-process retry	A momentary blip: a brief timeout, a deadlock, a connection reset.	Retry two or three times with exponential backoff and jitter, within the same worker.

Tier	For what	Action
Retry topic	A dependency that is down but will return, such as a service restarting.	Republish to a delayed retry topic, commit the original offset, and let the partition keep moving.
Dead letter queue	A message that can never succeed: malformed payload, unknown event type, reference to a deleted entity.	Publish to the DLQ with the error, stack trace and original metadata, commit, and alert. A human decides what to do.

The important consequence. In all three tiers the offset eventually advances. A permanently bad record is removed from the main flow rather than left to block it, and no data is lost because the message itself is preserved in the DLQ with enough context to replay it later once the bug is fixed.

3.13 Observability — knowing it is healthy

A streaming pipeline fails quietly, so the metrics are part of the design rather than an afterthought.

Signal	What it tells you	What action it triggers
Consumer lag per partition	Whether consumers are keeping up. Flat is healthy; steadily climbing is not.	Scale out up to the partition count, or investigate a slow dependency.
Rebalance rate	How often the group is reshuffling. Should be near zero outside deployments.	Any background rate means the poll loop or the timeouts need attention.
Processing time per record	Where time is actually going.	Tune the pool size or the slow dependency it waits on.
Duplicate-skip count	How often idempotency is catching a repeat. A small number is normal and healthy.	A sudden spike points at instability or an unnecessary replay.
DLQ arrival rate	Messages that could not be processed at all.	Alert immediately; each one is a bug or a bad producer.
Uneven lag across partitions	A hot key sending disproportionate traffic to one lane.	Revisit the partition key.

4. The Workflow, End to End

This section follows one real event from the moment it happens to the moment its offset is committed, then covers the four situations where things do not go to plan.

4.1 The happy path, step by step

Scenario: step 3 of workflow wf-77 has just finished successfully.

1. **Something happens.** The task executor finishes step 3 of workflow wf-77 and updates its own database to record that the step is complete.
2. **An event is created.** In the same database transaction it writes an event into an outbox table: eventType STEP_COMPLETED, workflowId wf-77, a freshly generated eventId, and a timestamp. Because the state change and the event are written together, they cannot disagree.
3. **The event is published.** A relay reads the outbox and calls the Kafka producer with key = wf-77.
4. **A partition is chosen.** The producer hashes the key and picks a partition — say partition 1. Every event for wf-77 has always gone here and always will, which is what keeps this workflow's events in order.
5. **Kafka stores and replicates it.** The leader broker for partition 1 appends the record at the next offset, say 4,207, and copies it to two follower brokers. Only then does it acknowledge the write, because the producer requires acks=all.
6. **The write is confirmed.** The producer receives the acknowledgement and the relay marks the outbox row as sent. The producer's job is finished; it does not know or care who will read the event.
7. **A consumer polls.** Consumer B owns partition 1. Its poll loop calls poll(200ms) and receives a batch of up to 100 records, including offset 4,207. The loop's in-memory position advances immediately, but nothing has been committed yet.
8. **The batch is handed off.** The loop submits the records to the bounded worker pool and returns to poll() straight away. It does not wait for any of them. This is what keeps the consumer visibly alive.
9. **A worker picks the record up.** A pool thread takes offset 4,207. Records from partition 1 are routed so that only one worker handles this partition's records at a time, preserving order.
10. **Idempotency is checked.** The worker asks whether this eventId is already in the processed-events store. It is not, so processing continues. Had it been present, the worker would have skipped straight to marking the offset done.
11. **The work is done atomically.** In one database transaction the worker inserts the eventId, advances the workflow state to reflect that step 3 is complete, writes an audit row, and queues a notification. Either all of it happens or none of it does.
12. **Completion is recorded.** The worker marks offset 4,207 as finished in the consumer's in-flight tracker for partition 1.
13. **The watermark is computed.** On its next pass the poll loop asks the tracker for the highest offset with nothing unfinished beneath it. If 4,205 through 4,207 are all done, the watermark is 4,207.
14. **The offset is committed.** The loop commits 4,208 as the next position to read for partition 1. Only now is this event officially finished. If the process died a millisecond earlier, the record would simply be delivered again and step 10 would recognise it.

15. **Downstream systems react.** Independent consumer groups on the same topic — analytics, notifications, the audit service — read the very same record and build their own views from it, at their own pace and with their own offsets, without affecting this consumer at all.

16. **The loop continues.** `poll()` is called again and the cycle repeats, typically many times per second.

4.2 What happens when a consumer crashes mid-batch

Consumer B is killed after processing offsets 4,205 to 4,207 but before committing them.

- Consumer B stops sending heartbeats. After `session.timeout.ms` the group coordinator declares it gone.
- A rebalance is triggered and partition 1 is reassigned to Consumer A.
- Consumer A reads the last committed offset for partition 1, which is 4,205 because the later commit never happened.
- Consumer A begins reading from 4,205 and redelivers three records that were in fact already processed.
- For each of them the idempotency check finds the `eventId` already present, so each is skipped without side effects.
- Consumer A commits 4,208 and continues normally. Nothing was lost, nothing was applied twice, and the only cost was a few wasted lookups.

4.3 What happens during a deployment

A rolling deployment restarts each consumer in turn, so every restart causes a rebalance. This is the routine case the listener exists for.

- The instance being restarted receives the `partitions-revoked` callback before it loses anything.
- It stops accepting new records for those partitions, then waits for in-flight work to drain, bounded by a timeout so shutdown cannot hang.
- It commits the resulting watermark synchronously, so the handover point is as far forward as it can honestly be.
- Cooperative sticky assignment moves only the partitions that must move; consumers that are unaffected keep working throughout.
- The new owner reads the freshly committed offset and resumes from there. Duplicates are limited to whatever genuinely could not finish in the drain window.

4.4 What happens when a record cannot be processed

- A transient error, such as a database timeout, is retried two or three times inside the worker with exponential backoff and jitter. Most failures end here.
- If it still fails, the record is republished to a delayed retry topic and the original offset is committed, so the partition keeps flowing while the dependency recovers.
- If the retry tier is exhausted, or the failure is clearly permanent — a malformed payload, an unknown event type — the record goes to the dead letter queue together with the error, the stack trace and the original metadata.
- The offset is committed and an alert fires. A human investigates, fixes the cause, and replays the DLQ message when it can succeed. Nothing behind it was ever blocked.

4.5 What happens when the same event is published twice

A producer sends an event, the network drops the acknowledgement, and the producer retries. Two copies now exist.

- Because the producer reuses the same eventId on retry rather than generating a new one, both copies are recognisably the same event.
- Both share the same key, so both land on the same partition and are read by the same consumer in order.
- The first is processed normally and its eventId is recorded. The second finds that eventId already present and is skipped.
- The duplicate-skip metric increments, which is the intended, observable behaviour rather than a silent swallow. Idempotent producer settings reduce how often this happens at all, but the consumer-side check is what makes it safe.

5. Architecture Diagram (Figure 1)

The diagram on the next page shows the whole system from the services that create events down to the systems that consume them. Section 5.1 explains every element in it.

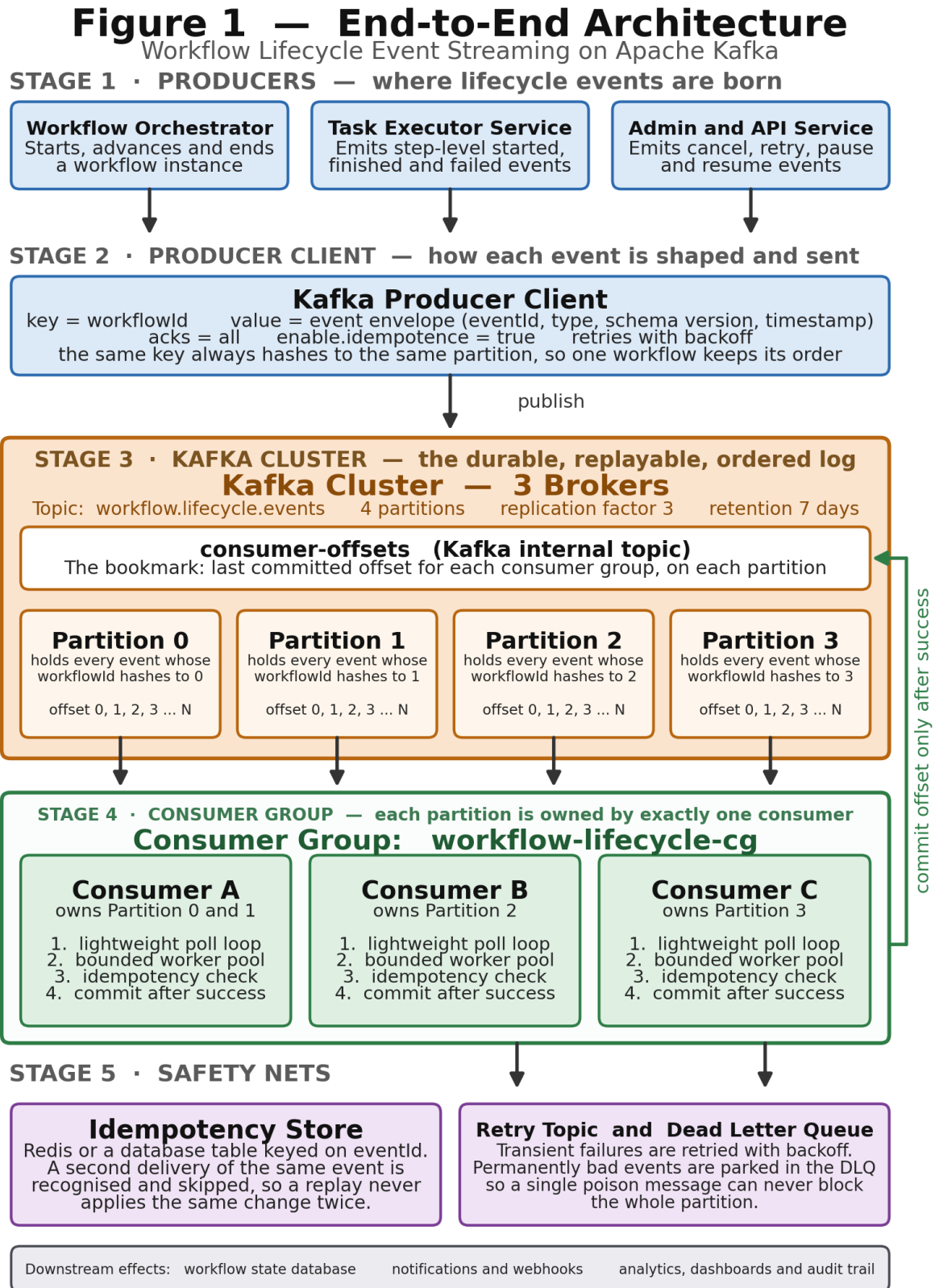


Figure 1 — End-to-end architecture of the workflow lifecycle event stream.

5.1 Reading the diagram, element by element

Element in Figure 1	What it represents and why it is drawn there
The three blue boxes at the top (Stage 1)	The producer services. They sit at the top because everything in the system starts with a real event happening inside one of them. They never talk to the consumers directly — that is the whole point of the design.
The single wide blue box (Stage 2)	The producer client library shared by all three services. It is drawn as one box because all three configure it identically: the same key choice, the same acknowledgement setting, the same idempotent-producer flag. This is where the partition decision is made.
The downward arrow labelled "publish"	The only direction data flows into Kafka. There is no reverse arrow here because a producer never waits for a consumer.
The large orange box (Stage 3)	The Kafka cluster itself — three brokers holding the topic. Everything inside this box is storage; nothing inside it processes business logic.
The line naming the topic	The topic's configuration: four partitions, replication factor three, seven-day retention. These three numbers determine parallelism, fault tolerance and how far back you can replay.
The four "Partition" boxes	The parallel lanes. Each shows that it holds events whose workflow ID hashes to that lane, and that records inside it are numbered by offset. Drawing them side by side emphasises that order is guaranteed down a lane, never across lanes.
The white "consumer-offsets" box	Kafka's internal bookkeeping topic. It is drawn inside the cluster because it is stored in Kafka like any other topic, and it holds the group's resume point for each partition.
The four arrows from partitions to consumers	Partition assignment. Each arrow ends at exactly one consumer, which is the visual statement of the rule that two consumers in a group never share a partition.
The green box (Stage 4)	The consumer group. Everything inside it is one logical reader made of several processes.
The three "Consumer" boxes	The individual instances and which partitions each owns. The 2-1-1 split shows that partitions need not divide evenly. The numbered list inside each box is the same four-step cycle every instance runs, expanded in Figure 2.
The green arrow up the right-hand side	The commit path. It flows from the consumers back into consumer-offsets, and it is labelled "only after success" because the entire correctness of the system depends on that ordering.
The two purple boxes (Stage 5)	The safety nets. The idempotency store makes redelivery harmless; the retry topic and DLQ keep a bad record from blocking a lane. They are drawn after processing because both act on the consequences of processing.
The grey bar at the bottom	The real-world effects the stream ultimately produces: the workflow state database, notifications, and the analytics and audit trail. These are the reason the pipeline exists.

6. Consumer Internals (Figure 2)

Figure 1 shows each consumer as a single box. Figure 2 opens that box, because the mechanism inside it is where the rebalance problem was actually solved.

Figure 2 — Inside One Consumer

The lightweight poll loop, the bounded worker pool and the commit cycle

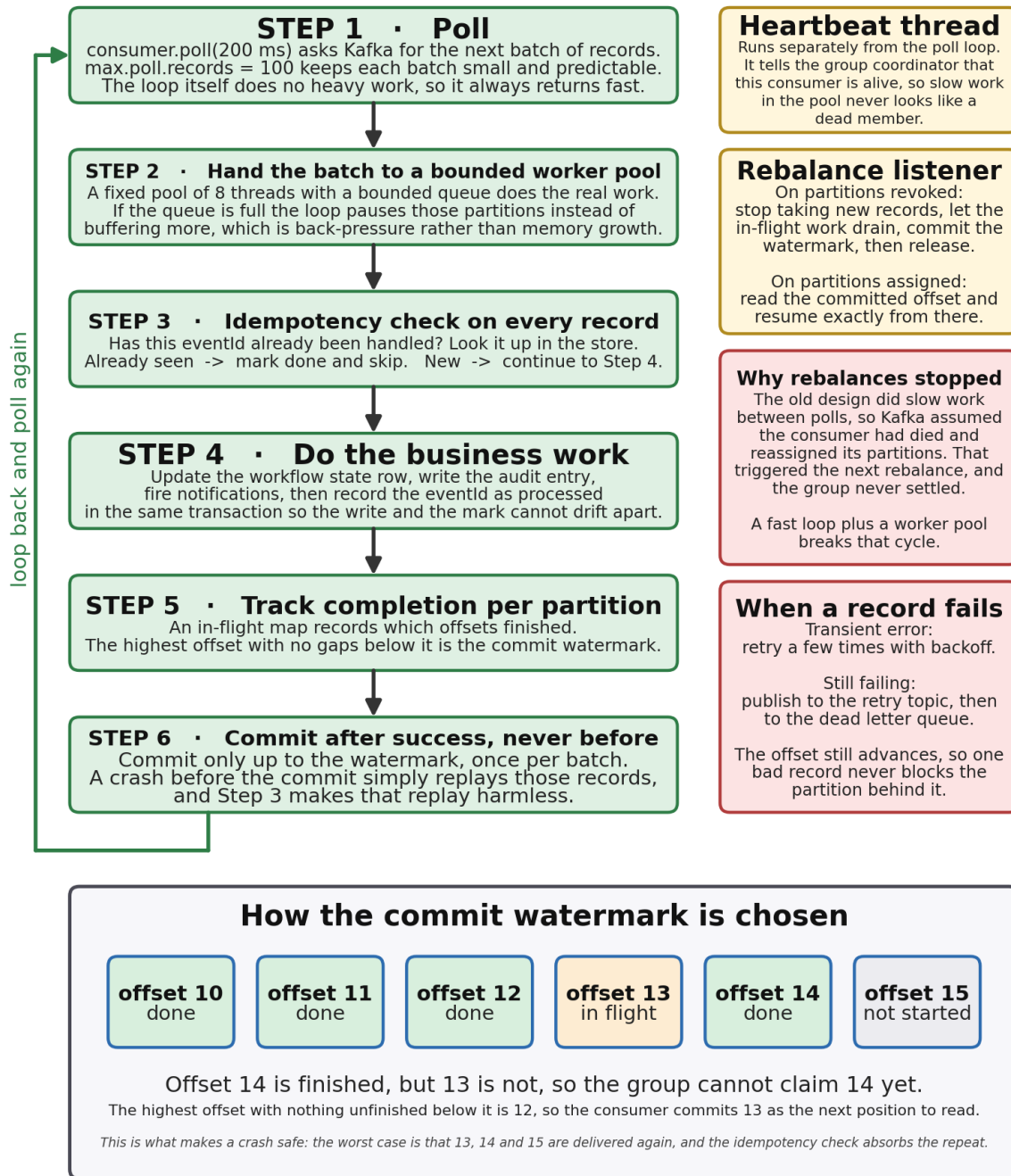


Figure 2 — Inside one consumer: the poll loop, the bounded worker pool and the commit cycle.

6.1 Reading the diagram, element by element

Element in Figure 2	What it represents and why it matters
The six green boxes down the left	One full cycle of the consumer, in order. The cycle is drawn as a loop rather than a line because it never ends while the consumer is running.
Step 1 — Poll	Fetching a bounded batch. The note that the loop does no heavy work is the single most important sentence in the diagram: <code>poll()</code> is also the liveness signal, so it must always return quickly.
Step 2 — Hand to the worker pool	The separation of concerns that makes the rest work. Handing off is fast; processing is slow; keeping them on different threads is what stops slow processing from looking like a dead consumer.
Step 3 — Idempotency check	Drawn before the business work, not after, because a check that runs afterwards cannot prevent a duplicate side effect.
Step 4 — Do the business work	The only step that changes the outside world. Recording the <code>eventId</code> in the same transaction as the state change is what keeps the record and the reality in agreement.
Step 5 — Track completion	The bookkeeping that concurrency forces on you. Because workers finish out of order, you cannot simply commit the newest offset you have seen.
Step 6 — Commit after success	The commit, deliberately last. Everything above it can be repeated safely; nothing above it may be skipped.
The arrow looping back on the left	The loop returning to <code>poll()</code> . It closes the cycle and reinforces that polling is continuous, not occasional.
The yellow "Heartbeat thread" box	A separate thread that maintains group membership. It is drawn outside the main column because it runs independently of the loop and does not wait for processing.
The yellow "Rebalance listener" box	The two callbacks. Revoked: drain, commit, release. Assigned: read the committed offset and resume. This is what makes a deployment cheap instead of disruptive.
The red "Why rebalances stopped" box	The failure that was fixed, stated next to the fix. Slow work between polls looked like death, eviction caused a rebalance, and the rebalance made everything slower — a loop that fed itself.
The red "When a record fails" box	The failure path, drawn beside the main flow because it is an exception, not a stage. The closing line — the offset still advances — is why one bad record cannot block a partition.
The offset strip at the bottom	A worked example of the watermark rule. Offset 14 is finished but 13 is not, so the group may only claim 12 and commits 13 as its next read position. It shows concretely why the commit is about a contiguous position, not about individual records.

7. The Rebalance Problem and How It Was Solved

This was the headline difficulty of the project, so it is worth stating the cause and the cure side by side.

7.1 The self-feeding loop

The original consumer processed each batch inline before polling again. On a slow batch the gap between polls exceeded `max.poll.interval.ms`, the coordinator concluded the consumer had died, and its partitions were reassigned. That reassignment paused every other consumer in the group, which made them slower, which caused the next one to miss its deadline. Meanwhile the evicted consumer's in-flight work was abandoned uncommitted, so the new owner replayed it, adding still more load. The group spent its time reshuffling rather than working, and it got worse precisely when traffic was heaviest.

7.2 Cause and cure

Symptom	Underlying cause	What fixed it
Consumers evicted while healthy	Heavy processing between poll calls exceeded the poll interval, so liveness could not be proved.	The lightweight poll loop. Processing moved to a worker pool; the loop returns in milliseconds.
Rebalances triggering more rebalances	Each reassignment paused the group, slowing everyone and pushing the next member over its deadline.	A fast loop plus cooperative sticky assignment, so only the partitions that must move are moved.
Memory growth and long GC pauses	Records were accepted faster than they could be processed, with no upper bound.	A bounded queue and a fixed thread pool, with <code>pause()</code> and <code>resume()</code> applying back-pressure.
Large bursts of duplicate work	Partitions were revoked mid-batch and in-flight work was abandoned without a commit.	A rebalance listener that drains in-flight work and commits the watermark before releasing.
Occasional lost updates	Offsets were committed before the work was known to have finished.	Auto-commit disabled and commit-after-success adopted, so unfinished work is always redelivered.
Duplicates causing real damage	Redelivery applied the same state change more than once.	An <code>eventId</code> -keyed idempotency check inside the same transaction as the business write.
One bad message freezing a lane	The record never succeeded and never committed, blocking everything behind it in an ordered partition.	Tiered retries and a dead letter queue, so the offset always advances.

The pattern behind all of it: keep the thread that proves you are alive separate from the thread that does the slow work, bound everything that can grow, and only ever declare progress that has actually been made.

8. Configuration Reference

Typical values for a pipeline of this shape, with the reason for each. Exact numbers depend on message size and how slow the downstream dependencies are.

8.1 Producer

Setting	Value	Why
acks	all	A write is acknowledged only once the replicas have it, so a broker failure cannot lose data.
enable.idempotence	true	Kafka de-duplicates producer retries at the broker, preventing duplicates caused by a lost acknowledgement.
max.in.flight.requests	5	Safe to keep high while idempotence is on; ordering is still preserved per partition.
retries	high, with backoff	Rides out brief broker or network problems rather than failing the caller.
key	workflowId	Forces all events for one workflow into one partition, which is what guarantees their order.
compression.type	snappy or lz4	Smaller batches mean less network and disk use, at negligible CPU cost.

8.2 Consumer

Setting	Value	Why
enable.auto.commit	false	A timer cannot know whether the work finished. Commits must be explicit and must follow success.
max.poll.records	100	Caps the size of a batch so a single poll cycle stays short and predictable.
max.poll.interval.ms	300000	The eviction deadline. With a lightweight loop the real gap is milliseconds, leaving a wide margin.
session.timeout.ms	45000	How long the coordinator waits for a heartbeat before declaring the member dead.
heartbeat.interval.ms	3000	Roughly a third of the session timeout, so a couple of lost heartbeats are tolerated.
partition.assignment.strategy	CooperativeSticky	Moves only the partitions that must move, instead of stopping the whole group.
auto.offset.reset	earliest	A brand-new group starts from the beginning of retention rather than silently skipping history.
isolation.level	read_committed	Only transactionally committed records are visible, where transactional producers are used.

8.3 Application-level

Setting	Typical value	Why
Worker pool threads	8	Sized to the database connection pool, which is the real bottleneck, not the CPU.
Bounded queue capacity	500	Large enough to absorb a burst, small enough that a stall is visible as back-pressure rather than memory growth.
Pause threshold	queue 80% full	Pausing before the queue is completely full leaves room to absorb the records already in flight.
Idempotency key TTL	longer than topic retention	A duplicate can only arrive from within the retention window, so the record must outlive it.
In-process retries	3, exponential with jitter	Absorbs transient failures; jitter stops every worker retrying in unison.
Drain timeout on revoke	a few seconds	Long enough for most in-flight work to finish, short enough that shutdown never hangs.

9. Design Rules Worth Remembering

If everything else is forgotten, these are the ideas that carry across to any streaming system.

1. Order is a per-key property, not a global one.

You almost never need global ordering. You need events about the same thing to stay in sequence, and choosing the right key gives you that while still allowing full parallelism across keys.

2. The poll call is a liveness signal, not just a fetch.

Any design that does slow work between polls will eventually be evicted for being slow. Separate the loop from the work.

3. Commit after success, never before.

Committing first risks losing events forever. Committing last risks duplicates, and duplicates can be neutralised. Choose the failure mode you can defend against.

4. At-least-once plus idempotency equals exactly-once in effect.

True exactly-once delivery is expensive and rarely necessary. Redelivery with a duplicate check gives the same business outcome far more simply.

5. Bound everything that can grow.

Unbounded queues and unbounded pools convert a slowdown into a crash. A bound turns it into visible back-pressure instead.

6. Assume every consumer will be restarted at the worst moment.

Deployments, autoscaling and crashes all happen mid-batch. Design the handover — drain, commit, release — rather than hoping it does not.

7. A message that can never succeed must be moved aside.

In an ordered partition, one poison record blocks everything behind it. Retry tiers and a dead letter queue keep the lane flowing.

8. Lag is the health metric.

It answers the only question that matters — are we keeping up — and it responds to every failure mode, from a slow dependency to a stuck consumer.

End of document